



**Linux
Professional
Institute**

Thema 101: Systemarchitektur



101.1 Hardwareeinstellungen ermitteln und konfigurieren

Referenz zu den LPI-Lernzielen

LPIC-1 v5, Exam 101, Objective 101.1

Gewichtung

2

Hauptwissensgebiete

- integrierte Peripheriegeräte aktivieren und deaktivieren
- die verschiedenen Arten von Massenspeichern unterscheiden
- Hardwareressourcen für Geräte ermitteln
- Werkzeuge und Hilfsprogramme, um verschiedene Hardwareinformationen aufzulisten (z. B. `lsusb`, `lspci` usw.)
- Werkzeuge und Hilfsprogramme, um USB-Geräte zu beeinflussen
- konzeptuelles Verständnis von `sysfs`, `udev`, `dbus`

Auszugsweise Liste der verwendeten Dateien, Begriffe und Hilfsprogramme

- `/sys/`
- `/proc/`
- `/dev/`
- `modprobe`
- `lsmod`
- `lspci`
- `lsusb`



Linux
Professional
Institute

101.1 Lektion 1

Zertifikat:	LPIC-1
Version:	5.0
Thema:	101 Systemarchitektur
Lernziel:	101.1 Hardwareeinstellungen ermitteln und konfigurieren
Lektion:	1 von 1

Einführung

In den letzten Jahrzehnten haben Hersteller von Großrechnern und Personalcomputern unzählige Hardwarekomponenten in ihre Maschinen integriert, die wiederum vom Betriebssystem unterstützt werden müssen. Dies würde Betriebssystementwickler vor eine unlösbare Aufgabe stellen, wenn die Industrie keine Standards für Befehlssätze und Gerätekommunikation festgelegt würde. Ähnlich wie die standardisierte Abstraktionsschicht, die das Betriebssystem einer Anwendung zur Verfügung stellt, erleichtern diese Standards die Entwicklung und Wartung von Betriebssystemen, die nicht an bestimmte Hardwaremodelle gebunden sind. Die Komplexität der integrierten Hardware erfordert jedoch manchmal Anpassungen hinsichtlich der Art und Weise, wie die Ressourcen dem Betriebssystem zugeteilt werden, damit sich dieses installiert lässt und korrekt funktioniert.

Einige dieser Anpassungen sind auch ohne ein installiertes Betriebssystem möglich. Die meisten Computer bieten ein Konfigurationsprogramm, das beim Einschalten der Maschine aufgerufen werden kann. Bis in die Mitte der 2000er Jahre war das Konfigurationsprogramm im BIOS (*Basic Input/Output System*) implementiert, einem Firmwarestandard, der typische Konfigurationsroutinen für x86-Hauptplatinen enthält. Ab dem Ende der 2000er Jahre wurde auf

x86-Maschinen das BIOS allmählich durch einen neuen Standard namens UEFI (*Unified Extensible Firmware Interface*) ersetzt. Dieser bietet mächtigere Funktionen für die Erkennung, Funktionsprüfung und Konfiguration von Hardware sowie die Aktualisierung von Firmware. Trotz der Umstellung auf UEFI spricht man in der Umgangssprache weiterhin vom BIOS, da beide Systeme dem gleichen Zweck dienen.

NOTE

Auf die Gemeinsamkeiten und Unterschiede zwischen BIOS und UEFI werden wir in einer späteren Lektion näher eingehen.

Aktivierung von Geräten

Das Systemkonfigurationsprogramm wird kurz nach dem Einschalten des Computers angezeigt, wenn man eine bestimmte Taste drückt. Welche Taste zu drücken ist, hängt vom Hersteller ab. Meistens handelt es sich um `Entf` oder eine der Funktionstasten, wie z.B. `F2` oder `F12`. Bei vielen Computern wird nach dem Einschalten ein Hinweis angezeigt, welche Taste oder Tastenkombination zu drücken ist.

In den BIOS-Einstellungen können integrierte Peripheriegeräte ein- und ausgeschaltet, grundlegende Schutzmechanismen aktiviert und Hardwareeinstellungen wie IRQ (*Interrupt Request*) und DMA (*Direct Memory Access*) geändert werden. Solche Änderungen sind bei modernen Computern in der Regel nicht notwendig. In Ausnahmefällen kann es jedoch sinnvoll sein, bestimmte Anpassungen vorzunehmen. Es gibt z.B. RAM-Technologien, die mit schnelleren Datenübertragungsraten als den Standardwerten kompatibel sind, daher wird empfohlen, diese auf die vom Hersteller angegebenen Werte einzustellen. Einige CPUs bieten Funktionen, die für den vorgesehenen Einsatzzweck eventuell nicht erforderlich sind und daher deaktiviert werden können. Dadurch wird der Stromverbrauch verringert. Auch die Sicherheit des Systems kann verbessert werden, wenn man z.B. CPU-Funktionen ausschaltet, die bekannte Sicherheitslücken enthalten.

Verfügt der Computer über mehrere Speichergeräte, ist es wichtig, die Bootreihenfolge im BIOS korrekt einzustellen. Das Gerät, das den Bootloader enthält, sollte an erster Stelle stehen. Ist dies nicht der Fall, wird das Betriebssystem unter Umständen nicht geladen.

Anzeige von Geräteinformationen in Linux

Sobald die Geräte korrekt identifiziert sind, obliegt es dem Betriebssystem, ihnen die jeweils benötigten Softwarekomponenten zuzuweisen. Wenn ein Gerät nicht erwartungsgemäß funktioniert, ist es wichtig zu unterscheiden, wo genau das Problem liegt. Wird eine Hardwarekomponente vom Betriebssystem gar nicht erst erkannt, ist es sehr wahrscheinlich, dass das Gerät—oder der Steckplatz, an den es angeschlossen ist—defekt ist. Wird eine

Hardwarekomponente zwar erkannt, funktioniert aber nicht richtig, liegt wahrscheinlich ein Problem seitens des Betriebssystems vor. Daher ist bei Hardwareproblemen immer zu prüfen, ob das Betriebssystem das Gerät korrekt erkennt. Es gibt im Wesentlichen zwei Wege, die von Linux erkannte Hardware anzuzeigen: Der erste Weg führt über geeignete Befehle, der zweite über Dateien in besonderen Dateisystemen.

Diagnostische Befehle

Die beiden grundlegenden Linux-Befehle für die Identifizierung von Geräten sind:

lspci

Zeigt alle Geräte an, die aktuell an den PCI-Bus (*Peripheral Component Interconnect*) angeschlossen sind. Bei PCI-Geräten handelt es sich entweder um Komponenten, die an die Hauptplatine angeschlossen sind (z.B. Festplatten-Controller), oder um Erweiterungskarten, die in einen PCI-Steckplatz eingesteckt ist (z.B. eine externe Grafikkarte).

lsusb

Listet USB-Geräte (*Universal Serial Bus*) auf, die aktuell an den Computer angeschlossen sind. Die USB-Schnittstelle hat ein weites Anwendungsspektrum, am häufigsten wird sie jedoch für den Anschluss von Eingabegeräten (z.B. Tastaturen, Zeigegeräte) und Wechselspeichermedien verwendet.

Die Ausgabe der Befehle `lspci` und `lsusb` besteht aus einer Liste aller vom Betriebssystem erkannten PCI- und USB-Geräte. Es kann jedoch sein, dass ein angezeigtes Gerät noch nicht voll funktionsfähig ist, da jedes Gerät eine Softwarekomponente benötigt, mit der es gesteuert wird. Diese Softwarekomponente wird als *Kernelmodul* bezeichnet und kann Teil des offiziellen Linux-Kernels sein oder separat aus anderen Quellen bezogen werden.

Linux-Kernelmodule, die der Steuerung von Geräten dienen, werden auch als *Treiber* (*Driver*) bezeichnet, wie es auch bei anderen Betriebssystemen üblich ist. Treiber für Linux werden jedoch nicht immer vom Hersteller des Geräts geliefert. Während manche Hersteller ihre eigenen binären Treiber zur Verfügung stellen, die separat installiert werden müssen, werden andere Treiber von unabhängigen Entwicklern erstellt. Früher hatten Geräte, die zum Beispiel unter Windows funktionierten, oft kein entsprechendes Kernelmodul für Linux. Heutzutage bieten Linux-basierte Betriebssysteme eine breite Hardwareunterstützung und die meisten Geräte funktionieren problemlos.

Befehle, die unmittelbar auf die Hardware zugreifen, erfordern oft root-Rechte oder zeigen nur begrenzte Informationen an, wenn sie von normalen Benutzern aufgerufen werden. So kann es notwendig sein, sich als root anzumelden oder derartige Befehle mit `sudo` auszuführen.

Die folgende Ausgabe des Befehls `lspci` zeigt beispielhaft einige erkannte Geräte:

```
$ lspci
01:00.0 VGA compatible controller: NVIDIA Corporation GM107 [GeForce GTX 750 Ti] (rev a2)
04:02.0 Network controller: Ralink corp. RT2561/RT61 802.11g PCI
04:04.0 Multimedia audio controller: VIA Technologies Inc. ICE1712 [Envy24] PCI Multi-
Channel I/O Controller (rev 02)
04:0b.0 FireWire (IEEE 1394): LSI Corporation FW322/323 [TrueFire] 1394a Controller (rev 70)
```

Die Ausgabe von `lspci` kann etliche Zeilen umfassen, daher zeigen wir hier nur ausgewählte Abschnitte. Die hexadezimale Zahl am Anfang jeder Zeile ist die eindeutige Adresse des jeweiligen PCI-Geräts. Der Befehl `lspci` zeigt mehr Details über ein bestimmtes Gerät, wenn dessen Adresse mit der Option `-s` angegeben wird, begleitet von der Option `-v`:

```
$ lspci -s 04:02.0 -v
04:02.0 Network controller: Ralink corp. RT2561/RT61 802.11g PCI
    Subsystem: Linksys WMP54G v4.1
    Flags: bus master, slow devsel, latency 32, IRQ 21
    Memory at e3100000 (32-bit, non-prefetchable) [size=32K]
    Capabilities: [40] Power Management version 2
    kernel driver in use: rt61pci
```

Die Ausgabe enthält nun weitere Details zum Gerät an der Adresse `04:02.0`. Es handelt sich um einen Netzwerkadapter mit der internen Bezeichnung `Ralink corp. RT2561/RT61 802.11g PCI`. Unter `Subsystem` sind Marke und Modell des Geräts angegeben: `Linksys WMP54G v4.1`. Dies kann für Diagnosezwecke hilfreich sein.

Das zugeordnete Kernelmodul ist in der Zeile `kernel driver in use` angegeben. In diesem Fall handelt es sich um das Modul `rt61pci`. Aus den gegebenen Informationen lässt sich Folgendes schließen:

1. Das Gerät wurde erkannt.
2. Ein passendes Kernelmodul wurde geladen.
3. Das Gerät sollte einsatzbereit sein.

Eine weitere Möglichkeit zu überprüfen, welches Kernelmodul für das angegebene Gerät verwendet wird, bietet die Option `-k`, die in neueren Versionen von `lspci` vorhanden ist:

```
$ lspci -s 01:00.0 -k
01:00.0 VGA compatible controller: NVIDIA Corporation GM107 [GeForce GTX 750 Ti] (rev a2)
```

```
kernel driver in use: nvidia
kernel modules: nouveau, nvidia_drm, nvidia
```

Der Zeile `kernel driver in use: nvidia` können wir entnehmen, dass das vorliegende Gerät—eine Grafikkarte—vom Kernelmodul `nvidia` gesteuert wird. In der Zeile `kernel modules: nouveau, nvidia_drm, nvidia` sind alle in Frage kommenden Module aufgeführt.

Der Befehl `lsusb` ähnelt `lspci`, listet aber ausschließlich Informationen zum USB-Bus auf:

```
$ lsusb
```

```
Bus 001 Device 029: ID 1781:0c9f Multiple Vendors USBtiny
Bus 001 Device 028: ID 093a:2521 Pixart Imaging, Inc. Optical Mouse
Bus 001 Device 020: ID 1131:1001 Integrated System Solution Corp. KY-BT100 Bluetooth Adapter
Bus 001 Device 011: ID 04f2:0402 Chicony Electronics Co., Ltd Genius LuxeMate i200 Keyboard
Bus 001 Device 007: ID 0424:7800 Standard Microsystems Corp.
Bus 001 Device 003: ID 0424:2514 Standard Microsystems Corp. USB 2.0 Hub
Bus 001 Device 002: ID 0424:2514 Standard Microsystems Corp. USB 2.0 Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Angezeigt werden die verfügbaren USB-Kanäle und die daran angeschlossenen Geräte. Wie bei `lspci` liefert die Option `-v` eine ausführlichere Ausgabe. Ein bestimmtes Gerät kann ausgewählt werden, indem man dessen ID mit der Option `-d` angibt:

```
$ lsusb -v -d 1781:0c9f
```

```
Bus 001 Device 029: ID 1781:0c9f Multiple Vendors USBtiny
Device Descriptor:
  bLength                18
  bDescriptorType         1
  bcdUSB                 1.01
  bDeviceClass           255 Vendor Specific Class
  bDeviceSubClass         0
  bDeviceProtocol         0
  bMaxPacketSize0         8
  idVendor               0x1781 Multiple Vendors
  idProduct              0x0c9f USBtiny
  bcdDevice              1.04
  iManufacturer          0
  iProduct               2 USBtiny
  iSerial                0
  bNumConfigurations     1
```

Mit der Option `-t` zeigt `lsusb` die aktuelle USB-Gerätezuordnung als hierarchischen Baum an:

```
$ lsusb -t
/: Bus 01.Port 1: Dev 1, Class=root_hub, Driver=dwc_otg/lp, 480M
   |__ Port 1: Dev 2, If 0, Class=Hub, Driver=hub/4p, 480M
      |__ Port 1: Dev 3, If 0, Class=Hub, Driver=hub/3p, 480M
         |__ Port 2: Dev 11, If 1, Class=Human Interface Device, Driver=usbhid, 1.5M
         |__ Port 2: Dev 11, If 0, Class=Human Interface Device, Driver=usbhid, 1.5M
         |__ Port 3: Dev 20, If 0, Class=Wireless, Driver=btusb, 12M
         |__ Port 3: Dev 20, If 1, Class=Wireless, Driver=btusb, 12M
         |__ Port 3: Dev 20, If 2, Class=Application Specific Interface, Driver=, 12M
         |__ Port 1: Dev 7, If 0, Class=Vendor Specific Class, Driver=lan78xx, 480M
      |__ Port 2: Dev 28, If 0, Class=Human Interface Device, Driver=usbhid, 1.5M
      |__ Port 3: Dev 29, If 0, Class=Vendor Specific Class, Driver=, 1.5M
```

Eventuell sind nicht allen Geräten entsprechende Module zugeordnet. Denn bestimmte Geräte können von Anwendungen direkt angesprochen werden, ohne dass ein Kernelmodul dazwischengeschaltet ist. In jedem Fall enthält die Ausgabe von `lsusb -t` nützliche Informationen. Wenn ein Modul verwendet wird, erscheint sein Name gegen Ende der Zeile, z.B. `Driver=btusb`. `Class` gibt die allgemeine Gerätekategorie an, z.B. `Human Interface Device`, `Wireless`, `Vendor Specific Class`, `Mass Storage` usw. Kennt man die Bus- und Dev-Nummer eines Geräts, kann man mit `lsusb -s` den Gerätenamen anzeigen lassen:

```
$ lsusb -s 01:20
Bus 001 Device 020: ID 1131:1001 Integrated System Solution Corp. KY-BT100 Bluetooth Adapter
```

In einem Linux-System sind normalerweise zahlreiche Kernelmodule geladen. Um mit diesen zu interagieren, verwendet man am besten die Befehle des Pakets `kmod`. Damit lassen sich typische Verwaltungsaufgaben in Bezug auf Kernelmodule erledigen. Dazu gehören das Einfügen, Entfernen und Auflisten von Modulen, das Anzeigen von deren Eigenschaften sowie das Auflösen von Abhängigkeiten und Aliasen. So zeigt zum Beispiel der Befehl `lsmod` alle aktuell geladenen Module an:

```
$ lsmod
Module                Size  Used by
kvm_intel             138528  0
kvm                   421021  1 kvm_intel
iTCO_wdt              13480   0
iTCO_vendor_support   13419   1 iTCO_wdt
snd_usb_audio         149112   2
```



```

snd_hda_codec_realtek 51465 1
snd_ice1712           75006 3
snd_hda_intel         44075 7
arc4                  12608 2
snd_cs8427            13978 1 snd_ice1712
snd_i2c               13828 2 snd_ice1712,snd_cs8427
snd_ice17xx_ak4xxx    13128 1 snd_ice1712
snd_ak4xxx_adda       18487 2 snd_ice1712,snd_ice17xx_ak4xxx
microcode            23527 0
snd_usbmidi_lib       24845 1 snd_usb_audio
gspca_pac7302         17481 0
gspca_main            36226 1 gspca_pac7302
videodev             132348 2 gspca_main,gspca_pac7302
rt61pci              32326 0
rt2x00pci            13083 1 rt61pci
media                20840 1 videodev
rt2x00mmio           13322 1 rt61pci
hid_dr               12776 0
snd_mpu401_uart      13992 1 snd_ice1712
rt2x00lib            67108 3 rt61pci,rt2x00pci,rt2x00mmio
snd_rawmidi          29394 2 snd_usbmidi_lib,snd_mpu401_uart

```

Die Ausgabe von `lsmod` umfasst drei Spalten:

Module

Name des Moduls

Size

Arbeitsspeicher (in Bytes), der vom Modul belegt wird

Used by

Abhängige Module

Einige Module benötigen andere Module, um korrekt zu funktionieren. Dies ist z.B. bei Modulen für Audiogeräte der Fall:

```

$ lsmod | fgrep -i snd_hda_intel
snd_hda_intel      42658  5
snd_hda_codec     155748  3 snd_hda_codec_hdmi,snd_hda_codec_via,snd_hda_intel
snd_pcm           81999  3 snd_hda_codec_hdmi,snd_hda_codec,snd_hda_intel
snd_page_alloc    13852  2 snd_pcm,snd_hda_intel
snd               59132  19

```

```
snd_hwdep,snd_timer,snd_hda_codec_hdmi,snd_hda_codec_via,snd_pcm,snd_seq,snd_hda_codec,snd_hda_intel,snd_seq_device
```

Die dritte Spalte, `Used by`, zeigt die Module, die das Modul in der ersten Spalte benötigen, um ordnungsgemäß zu funktionieren. Viele Module der Linux-Soundarchitektur, deren Name mit `snd` beginnt, sind voneinander abhängig. Treten Probleme auf, kann es bei der Fehlersuche helfen, einzelne geladene Module probeweise zu entladen. Dies kann man mit dem Befehl `modprobe` tun, der sowohl dem Laden als auch dem Entladen von Kernelmodulen dient. Zum Entladen eines Moduls und damit zusammenhängender Module (sofern diese nicht von einem laufenden Prozess gebraucht werden) wird die Option `-r` verwendet. Um etwa das Modul `snd-hda-intel` (Treiber für HDA-Intel-Audiogeräte) und andere Module, die mit dem Soundsystem zusammenhängen, zu entladen, lautet der Befehl:

```
# modprobe -r snd-hda-intel
```

Bereits beim Laden des Kernels werden zahlreiche Module mitgeladen. Die Parameter, mit denen die Module geladen werden, lassen sich ändern, ähnlich wie man bei Befehlen unterschiedliche Optionen verwendet. Jedes Modul akzeptiert bestimmte Parameter, wobei in den meisten Fällen die Standardwerte genau richtig sind. Manchmal ist es jedoch notwendig, auf bestimmte Parameter zurückzugreifen, damit ein Modul erwartungsgemäß funktioniert.

Ruft man den Befehl `modinfo` mit einem Modulnamen als Argument auf, zeigt dieser eine Beschreibung, die Datei, den Autor, die Lizenz, die Identifikation, die Abhängigkeiten und die verfügbaren Parameter an. Möchte man Modulparameter auf Dauer festlegen, schreibt man diese in die Datei `/etc/modprobe.conf` oder in eine Datei mit der Erweiterung `.conf` im Verzeichnis `/etc/modprobe.d/`. Um mit `modinfo` nur die verfügbaren Parameter eines Moduls anzuzeigen, verwendet man die Option `-p`:

```
# modinfo -p nouveau
vram_pushbuf:Create DMA push buffers in VRAM (int)
tv_norm:Default TV norm.
        Supported: PAL, PAL-M, PAL-N, PAL-Nc, NTSC-M, NTSC-J,
        hd480i, hd480p, hd576i, hd576p, hd720p, hd1080i.
        Default: PAL
        NOTE Ignored for cards with external TV encoders. (charp)
nofbaccel:Disable fbcon acceleration (int)
fbcon_bpp:fbcon bits-per-pixel (default: auto) (int)
mst:Enable DisplayPort multi-stream (default: enabled) (int)
tv_disable:Disable TV-out detection (int)
ignorelid:Ignore ACPI lid status (int)
```

```

duallink:Allow dual-link TMDS (default: enabled) (int)
hdmimhz:Force a maximum HDMI pixel clock (in MHz) (int)
config:option string to pass to driver core (charp)
debug:debug string to pass to driver core (charp)
noaccel:disable kernel/abi16 acceleration (int)
modeset:enable driver (default: auto, 0 = disabled, 1 = enabled, 2 = headless) (int)
atomic:Expose atomic ioctl (default: disabled) (int)
runpm:disable (0), force enable (1), optimus only default (-1) (int)

```

Die Ausgabe zeigt alle verfügbaren Parameter für das Modul `nouveau`, ein Kernelmodul, das vom *Nouveau Project* als Alternative zu den proprietären NVIDIA-Treibern angeboten wird. Mit der Option `modeset` steuern Sie beispielsweise, ob Bildschirmauflösung und -tiefe im Kernelspace statt im Userspace gesetzt werden. Indem man `options nouveau modeset=0` in die Datei `/etc/modprobe.d/nouveau.conf` schreibt, deaktiviert man die Modeset-Kernelfunktion.

Verursacht ein Modul Probleme, kann man die Datei `/etc/modprobe.d/blacklist.conf` editieren, um das Laden des Moduls zu verhindern. Um zum Beispiel das automatische Laden des Moduls `nouveau` zu verhindern, fügt man die Zeile `blacklist nouveau` in der Datei `/etc/modprobe.d/blacklist.conf` ein. Dies kann erforderlich sein, wenn das proprietäre Modul `nvidia` installiert ist und das Standardmodul `nouveau` nicht verwendet werden soll.

NOTE

Sie können die Datei `/etc/modprobe.d/blacklist.conf` editieren, die in vielen Linux-Distributionen standardmäßig vorhanden ist. Die empfohlene Methode ist jedoch, eine separate Konfigurationsdatei `/etc/modprobe.d/<modulename>.conf` zu erstellen, die Einstellungen speziell für das angegebene Kernelmodul enthält.

Informationsdateien und Gerätedateien

Die Befehle `lspci`, `lsusb` und `lsmod` bieten einen benutzerfreundlichen Weg, die vom Betriebssystem gespeicherten Hardwareinformationen abzurufen. Diese Informationen befinden sich in speziellen Dateien in den Verzeichnissen `/proc` und `/sys`. Die beiden Verzeichnisse verweisen auf spezielle Dateisysteme, die nicht auf Speichergeräten liegen, sondern in einem RAM-Bereich, in dem der Kernel Konfigurationseinstellungen sowie Informationen über laufende Prozesse ablegt. Solche Dateisysteme sind nicht für die normale Dateispeicherung gedacht, daher werden sie Pseudo-Dateisysteme genannt. Sie existieren nur, solange das System läuft. Das Verzeichnis `/proc` enthält Dateien mit Informationen über laufende Prozesse und Hardwareressourcen. Dazu gehören zum Beispiel:

`/proc/cpuinfo`

Enthält genaue Informationen über die vom Betriebssystem erkannte(n) CPU(s).

/proc/interrupts

Enthält eine Tabelle mit der Anzahl der Unterbrechungen, die pro Eingabe/Ausgabe-Baustein für jede CPU aufgetreten sind.

/proc/ioports

Enthält eine Liste der aktuell registrierten Port-Bereiche für die Ein- und Ausgabe.

/proc/dma

Gibt Auskunft über die vom System verwendeten DMA-Kanäle (*Direct Memory Access*).

Die Dateien im Verzeichnis `/sys` haben einen ähnlichen Zweck wie jene in `/proc`. Dabei enthält `/sys` vor allem Geräteinformationen und Hardwaredaten des Kernels, während `/proc` eigentlich Kerneldatenstrukturen abbildet, zu denen auch Daten über die laufenden Prozesse gehören.

Ein weiteres Standardverzeichnis unter Linux, das sich auf die Systemhardware bezieht, ist `/dev`. Jede Datei in `/dev` stellt ein Gerät dar. Dazu gehören insbesondere Speichergeräte. So wird zum Beispiel eine an den ersten IDE-Kanal der Hauptplatine angeschlossene Festplatte durch die Datei `/dev/hda` repräsentiert. Die einzelnen Partitionen dieser Festplatte sind dabei als `/dev/hda1`, `/dev/hda2` usw. dargestellt.

Wechselmedien werden vom Subsystem `udev` verwaltet, das die entsprechenden Gerätedateien in `/dev` erstellt. Der Linux-Kernel erkennt neu angeschlossene Hardware und übergibt die entsprechenden Informationen an den `udev`-Prozess, der dann das Gerät identifiziert und dynamisch entsprechende Dateien unter `/dev` anlegt, wobei vordefinierte Regeln zur Anwendung kommen.

In aktuellen Linux-Distributionen identifiziert und konfiguriert `udev` nicht nur die im laufenden Betrieb erkannten Geräte (*hotplug detection*), sondern auch die bereits beim Einschalten des Rechners vorhandenen Geräte (*coldplug detection*). `Udev` verwendet `SysFS`, das oben genannte Pseudo-Dateisystem für hardwarebezogene Informationen, das auch in `/sys` eingebunden ist.

NOTE

Unter Hotplug versteht man die Erkennung und Konfiguration eines Geräts im laufenden Betrieb, z.B. beim Einstecken eines USB-Geräts. Der Linux-Kernel unterstützt Hotplug-Erkennung seit Version 2.6. Die meisten Systembusse (PCI, USB usw.) können Hotplug-Ereignisse auslösen, wenn ein Gerät ein- oder ausgesteckt wird.

Wird ein neues Gerät erkannt, sucht `udev` im Verzeichnis `/etc/udev/rules.d/` nach einer passenden Regel dafür. Die wichtigsten Regeln werden von den Distributionen zur Verfügung gestellt. Für besondere Fälle können neue Regeln hinzugefügt werden.

Speichergeräte

In Linux werden Speichergeräte als Blockgeräte bezeichnet, da Daten auf diesen Geräten in Blöcken unterschiedlicher Größe und Position gespeichert werden. Jedes Blockgerät wird durch eine Datei im Verzeichnis `/dev` repräsentiert, wobei der Name der Datei vom Gerätetyp (IDE, SATA, SCSI usw.) und seinen Partitionen abhängt. So wird zum Beispiel ein CD/DVD-Laufwerk, das an den zweiten IDE-Kanal angeschlossen ist, als `/dev/hdc` abgebildet (`/dev/hda` und `/dev/hdb` sind für die Master- und Slave-Geräte des ersten IDE-Kanals reserviert), und ein altes Diskettenlaufwerk wird als `/dev/fd0`, `/dev/fd1` o.ä. dargestellt.

Seit Version 2.4 des Linux-Kernels werden die meisten Speichergeräte — unabhängig von ihrem Hardwaretyp — so gekennzeichnet, als wären sie SCSI-Geräte. Das heißt, IDE-, SSD- und USB-Blockgeräte werden mit dem Präfix `sd` bezeichnet. Bei IDE-Platten wird durch einen dritten Buchstaben markiert, ob das Laufwerk Master oder Slave ist (im ersten IDE-Kanal steht `sda` für Master und `sdb` für Slave). Partitionen werden der Reihe nach nummeriert. Zum Beispiel stehen die Pfade `/dev/sda1`, `/dev/sda2` usw. für die erste, zweite usw. Partition des zuerst identifizierten Blockgeräts, und `/dev/sdb1`, `/dev/sdb2` usw. für die erste, zweite usw. Partition des als zweites identifizierten Blockgeräts. Ausnahmen von diesem Muster bilden Speicherkarten (SD-Karten) und NVMe-Geräte (an den PCI-Express-Bus angeschlossene SSD-Geräte): Bei SD-Karten werden die Dateien `/dev/mmcblk0p1`, `/dev/mmcblk0p2` usw. für die Partitionen des zuerst erkannten Geräts verwendet, und `/dev/mmcblk1p1`, `/dev/mmcblk1p2` usw. bezeichnen die Partitionen des als zweites erkannten Geräts. NVMe-Geräte erhalten das Präfix `nvme`. Die Partitionen heißen dann z.B. `/dev/nvme0n1p1`, `/dev/nvme0n1p2` usw.

Geführte Übungen

1. Ein Betriebssystem kann nicht mehr hochfahren, nachdem der Computer um eine zweite SATA-Platte erweitert wurde. Angenommen, es liegt kein Hardwaredefekt vor: Was könnte die Ursache für diesen Fehler sein?

2. Sie möchten überprüfen, ob die Grafikkarte, die an den PCI-Bus Ihres neu erworbenen Computers angeschlossen ist, wirklich die vom Hersteller beworbene ist. Allerdings möchten Sie das Computergehäuse nicht öffnen, da dies zum Erlöschen der Garantie führen würde. Welchen Befehl können Sie verwenden, um die Spezifikationen der Grafikkarte anzuzeigen, wie sie vom Betriebssystem erkannt wurden?

3. Betrachten Sie die folgende Ausgabe von `lspci`:

```
03:00.0 RAID bus controller: LSI Logic / Symbios Logic MegaRAID SAS 2208 [Thunderbolt]
(rev 05)
```

Welchen Befehl können Sie ausführen, um das Kernelmodul zu identifizieren, das für dieses Gerät verwendet wird?

4. Sie möchten verschiedene Parameter für das Kernelmodul `bluetooth` ausprobieren, ohne das System neu zu starten. Der Versuch, das Modul mit `modprobe -r bluetooth` zu entladen, führt jedoch zu folgendem Fehler:

```
modprobe: FATAL: Module bluetooth is in use.
```

Was ist die wahrscheinlichste Ursache für diesen Fehler?

Offene Übungen

1. In Rechenzentren findet man bisweilen veraltete Computer vor, z.B. weil externe Geräte unterstützt werden müssen, die über eine veraltete Hardware-Schnittstelle kommunizieren. Bei diesen älteren Maschinen müssen einige Besonderheiten beachtet werden. Einige x86-Server mit älterer BIOS-Firmware booten zum Beispiel nicht, wenn keine Tastatur erkannt wird. Wie lässt sich dieses Problem vermeiden?

2. Linux-Betriebssysteme unterstützen nicht nur x86, sondern viele weitere Computerarchitekturen. Dazu gehört u.a. die ARM-Architektur, die auf vielen Einplatinencomputern zum Einsatz kommt. Sie werden feststellen, dass der Befehl `lspci` auf solchen Maschinen, wie beispielsweise dem Raspberry Pi, nicht zur Verfügung steht. Welcher Unterschied zu x86-Maschinen ist dafür verantwortlich?

3. Viele Netzwerkrouter verfügen über einen USB-Port, an den ein externes Gerät, wie z.B. eine USB-Festplatte, angeschlossen werden kann. Auf den meisten dieser Router läuft Linux als Betriebssystem. Wie wird Linux auf einem solchen Gerät eine externe USB-Festplatte im Verzeichnis `/dev/` benennen, wenn kein anderes konventionelles Blockgerät vorhanden ist?

4. Im Jahr 2018 wurde die als *Meltdown* bekannte Hardwareschwachstelle entdeckt. Sie betrifft unzählige Prozessoren verschiedener Architekturen. Neuere Versionen des Linux-Kernels informieren darüber, ob das vorliegende System für *Meltdown* anfällig ist. Wie kann diese Information abgerufen werden?

Zusammenfassung

In dieser Lektion wird grundlegend dargestellt, wie der Linux-Kernel Hardwareressourcen darstellt, vor allem auf x86-Rechnern. Dabei werden u.a. folgende Themen behandelt:

- Wie im BIOS- oder UEFI-Konfigurationsprogramm eingestellt werden kann, wie das Betriebssystem mit der Hardware interagiert.
- Wie man die von Linux bereitgestellten Tools verwendet, um Informationen über die Hardware zu erhalten.
- Wie interne und externe Speichergeräte im Dateisystem dargestellt sind.

Insbesondere behandelt werden:

- Befehle zur Sichtung der erkannten Hardware: `lspci` und `lsusb`.
- Befehle zur Verwaltung von Kernelmodulen: `lsmod` und `modprobe`.
- Hardwarebezogene Spezialdateien in den Verzeichnissen `/dev`, `/proc` und `/sys`.

Lösungen zu den geführten Übungen

1. Ein Betriebssystem kann nicht mehr hochfahren, nachdem der Computer um eine zweite SATA-Platte erweitert wurde. Angenommen, es liegt kein Hardwaredefekt vor: Was könnte die Ursache für diesen Fehler sein?

Die Reihenfolge der Bootgeräte sollte im BIOS-Setup konfiguriert werden, sonst kann das BIOS den Bootloader möglicherweise nicht ausführen.

2. Sie möchten überprüfen, ob die Grafikkarte, die an den PCI-Bus Ihres neu erworbenen Computers angeschlossen ist, wirklich die vom Hersteller beworbene ist. Allerdings möchten Sie das Computergehäuse nicht öffnen, da dies zum Erlöschen der Garantie führen würde. Welchen Befehl können Sie verwenden, um die Spezifikationen der Grafikkarte anzuzeigen, wie sie vom Betriebssystem erkannt wurden?

Der Befehl `lspci` listet detaillierte Informationen über alle Geräte auf, die an den PCI-Bus angeschlossen sind.

3. Betrachten Sie die folgende Ausgabe von `lspci`:

```
03:00.0 RAID bus controller: LSI Logic / Symbios Logic MegaRAID SAS 2208 [Thunderbolt]
(rev 05)
```

Welchen Befehl können Sie ausführen, um das Kernelmodul zu identifizieren, das für dieses Gerät verwendet wird?

Der Befehl `lspci -s 03:00.0 -v` oder `lspci -s 03:00.0 -k` liefert die gewünschte Information.

4. Sie möchten verschiedene Parameter für das Kernelmodul `bluetooth` ausprobieren, ohne das System neu zu starten. Der Versuch, das Modul mit `modprobe -r bluetooth` zu entladen, führt jedoch zu folgendem Fehler:

```
modprobe: FATAL: Module bluetooth is in use.
```

Was ist die wahrscheinlichste Ursache für diesen Fehler?

Das Modul `bluetooth` wird von einem laufenden Prozess verwendet.

Lösungen zu den offenen Übungen

1. In Rechenzentren findet man bisweilen veraltete Computer vor, z.B. weil externe Geräte unterstützt werden müssen, die über eine veraltete Hardware-Schnittstelle kommunizieren. Bei diesen älteren Maschinen müssen einige Besonderheiten beachtet werden. Einige x86-Server mit älterer BIOS-Firmware booten zum Beispiel nicht, wenn keine Tastatur erkannt wird. Wie lässt sich dieses Problem vermeiden?

Das BIOS-Konfigurationsprogramm verfügt über eine Option zum Deaktivieren der Sperre des Computers, wenn keine Tastatur gefunden wird.

2. Linux-Betriebssysteme unterstützen nicht nur x86, sondern viele weitere Computerarchitekturen. Dazu gehört u.a. die ARM-Architektur, die auf vielen Einplatinencomputern zum Einsatz kommt. Sie werden feststellen, dass der Befehl `lspci` auf solchen Maschinen, wie beispielsweise dem Raspberry Pi, nicht zur Verfügung steht. Welcher Unterschied zu x86-Maschinen ist dafür verantwortlich?

Im Gegensatz zu den meisten x86-Maschinen haben ARM-Computer wie der Raspberry Pi keinen PCI-Bus, so dass der Befehl `lspci` nicht benötigt wird.

3. Viele Netzwerkrouter verfügen über einen USB-Port, an den ein externes Gerät, wie z.B. eine USB-Festplatte, angeschlossen werden kann. Auf den meisten dieser Router läuft Linux als Betriebssystem. Wie wird Linux auf einem solchen Gerät eine externe USB-Festplatte im Verzeichnis `/dev/` benennen, wenn kein anderes konventionelles Blockgerät vorhanden ist?

Aktuelle Linux-Kernel behandeln USB-Festplatten wie SATA-Geräte, so dass die entsprechende Datei `/dev/sda` heißen wird.

4. Im Jahr 2018 wurde die als *Meltdown* bekannte Hardwareschwachstelle entdeckt. Sie betrifft unzählige Prozessoren verschiedener Architekturen. Neuere Versionen des Linux-Kernels informieren darüber, ob das vorliegende System für *Meltdown* anfällig ist. Wie kann diese Information abgerufen werden?

In der Datei `/proc/cpuinfo` gibt es für jede CPU eine Zeile `bugs:`, in der die bekannten Schwachstellen des Prozessors aufgelistet sind, z.B. `cpu_meltdown`.